PULSE Instrument Cluster

Ahmed Abdelghany

2026-09-03

Contents

Document control

Field	Value
Document ID	PULSE-SAD-001
Title	System Architecture Description
Version	1.1
Status	Baseline for sprint 1 review
Date	2026-09-03
Author	Ahmed Abdelghany
Reviewer	Cockpit Electronics / HMI Platform Group (sponsor)
Review gate	Sprint 1 review, week 3, 4 September 2026
Parent	PULSE-SyRS-001
Related	PULSE-StRS-001, PULSE-SAF-001, PULSE-SEC-001, PULSE-RTM-001

Change log

Version	Date	Author	Change
1.0	2026-09-03	A. Abdelghany	First baseline, written from the running system: context, components, interfaces, runtime, deployment, decisions and deviations, signal contract appendix
1.1	2026-09-03	A. Abdelghany	Layering, worked signal path (new section 5.4), runtime loop, manoeuvre state machine and deployment diagrams added; architecture principle diagram moved to draw.io

1. Purpose and scope

This document describes the architecture that satisfies PULSE-SyRS-001. It is the architecture baseline named in work package 01. It records what exists at week 3, what is planned, and the design decisions a later safety or security argument would need to know about (PULSE-SAF-001 SR-10).

It is a system-level description. Per-component software architecture (SWAD) is deferred to sprint 2 for the cluster and the providers.

Abbreviations used here are defined in the glossary, PULSE-StRS-001 section 8.

2. Architectural principle

Every component meets every other component only at the signal broker, and only through named signals from a standard catalogue. No component knows the others exist. Above the broker there is no CAN, no DBC and no hardware. Below the broker there is no user interface.

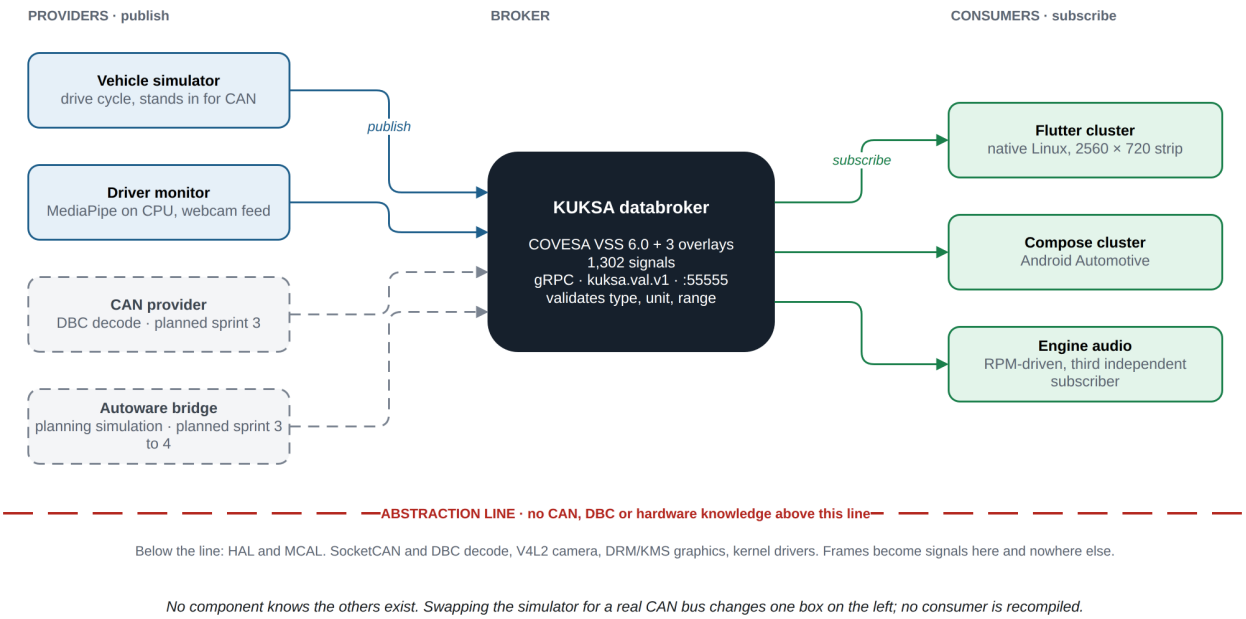

Figure 1: Architecture principle: providers publish, consumers subscribe, nothing meets anywhere but the broker.

Source: `docs/diagrams/pulse-architecture.drawio` (edit in draw.io, re-export with `scripts/build-docs.sh`).

Swapping the simulator for a real CAN bus changes one box on the left. No consumer is recompiled (FR-5, FR-8).

3. Context

3.1 System boundary

Inside the boundary	Outside the boundary
Signal catalogue and overlays	The vehicle and its buses (no vehicle exists on the desk rig)
Databroker	The driver (observed by the camera, addressed by the display)
Telemetry providers: simulator, driver monitor, later CAN feeder and Autoware bridge	The display panel hardware
Both HMI applications	The host operating system, container runtime and graphics stack
Orchestration scripts and the regression suite	The Android Automotive emulator or device

3.2 External interfaces

Interface	Direction	Technology	Notes
Camera	In	V4L2 (`/dev/video0`), 1280 x 720 at 30 fps	Frames never leave the driver-monitor process (CS-6)
Strip display	Out	DisplayPort alt mode over USB-C, 2560 x 720	Located at runtime by `xrandr`, placement via `CLUSTER_GEOM` (IR-4)
Vehicle bus	In	SocketCAN (`vcan0` then a CAN HAT on SPI)	Planned S3
Autonomy stack	In	Autoware planning simulation via ROS 2 bridge	Planned S3 to S4
Android surface	Out	ADB reverse to reach the broker on the host	Emulator today; device is backlog

4. Components

4.1 Component catalogue

Component	Role	Implementation	Provenance	Location
Signal catalogue	Names every signal, its type, unit, range and allowed values	COVESA VSS 6.0 plus AGL overlay plus two PULSE overlays, compiled with vss-tools to JSON	Upstream (VSS, AGL overlay), authored (PULSE overlays)	`vss/`
Databroker	Serves the tree, validates writes, fans out subscriptions	Eclipse KUKSA databroker 0.7.0, Rust, in a container	Upstream	`scripts/start-databroker.sh`
Vehicle simulator	Publishes a plausible, correlated drive cycle; runs the minimum-risk manoeuvre in cruise mode	Python, kuksa-client 0.5.2	Authored	`emulator/telemetry_sim.py`
Driver monitor	Face landmarks to fatigue, distraction and alert state; publishes derived state only	Python, MediaPipe FaceLandmarker (3.7 MB model, CPU), OpenCV	Authored (pipeline), upstream (model)	`emulator/dms.py`, `emulator/models/`
Driver monitor viewer	Debug window: landmarks, metrics, thresholds; can stand in for the monitor	Python, OpenCV	Authored	`emulator/dms_viewer.py`
Engine audio	RPM-driven engine sound, a third independent subscriber	Python	Authored	`emulator/engine_audio.py`

Component	Role	Implementation	Provenance	Location
Flutter cluster	The HMI on Linux; two screens (PULSE racing, classic analogue) in a swipeable pager	Flutter, Dart, custom-painted canvas; gRPC stubs generated from the KUKSA protos	Authored (UI, service), generated (stubs), scaffold (runner, one deliberate edit)	`pulse-cluster/`
Compose cluster	The same design on Android Automotive	Kotlin, Jetpack Compose, gRPC	Authored	`pulse-cluster-android/`
AGL reference cluster	AGL's own Flutter cluster, used as a comparison point	Flutter	Upstream, cloned from AGL Gerrit, not part of this repository	`flutter-instrument-cluster/`
Orchestration	Start and stop the stack, place the window, fall back to a normal window	Bash	Authored	`run.sh`, `scripts/`
Documentation	Onboarding, this specification set, traceability generator	Markdown, Python	Authored	`README.md`, `ONBOARDING.md`, `docs/`

The two HMIs share no code. They share only the signal contract in appendix A.

4.2 Layering

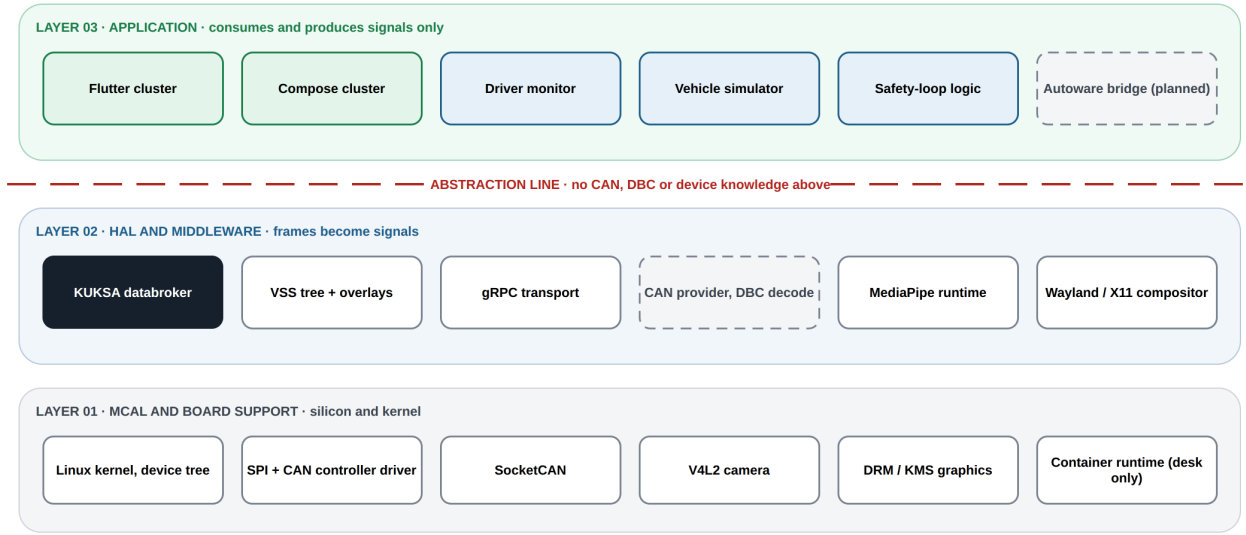

Figure 2: Three layers and the abstraction line. Nothing above the line knows a CAN frame, a DBC file or a device.

Source: `docs/diagrams/layering.drawio`.

Layer	Contents	Rule
03 Application	Flutter cluster, Compose cluster, driver monitor, safety-loop logic in the simulator, Autoware bridge, vehicle simulator	Consumes and produces signals only
Abstraction line		No CAN, DBC or device knowledge above this line
02 HAL and middleware	KUKSA databroker, VSS tree and overlays, gRPC transport, CAN provider with DBC decode, MediaPipe runtime, window system	Frames become signals here
01 MCAL and board support	Linux kernel and device tree, SPI and CAN controller driver, SocketCAN, V4L2 camera, DRM/KMS graphics	Silicon and kernel

5. Interfaces

5.1 Broker interface

Property	Value
Protocol	gRPC over HTTP/2
API	`kuksa.val.v1` (Get, Set, Subscribe streaming)
Address	`127.0.0.1:55555`, bound to loopback only
Transport security	None in the development configuration (`--insecure`); see PULSE-SEC-001
Authorisation	None in the development configuration; see PULSE-SEC-001 CS-4
Validation	Broker rejects a value whose datatype, `allowed` list or `min`/`max` does not match the catalogue (IR-2, CS-5)
Client stubs	Dart and Kotlin generated from the same `.proto` files; Python via kuksa-client

`kuksa.val.v1` was chosen over the newer `kuksa.val.v2` because AGL's reference stack and the AGL overlay target v1. Migration to v2 is a recorded decision, section 8.

5.2 Publish rates

Producer	Signals	Rate
Vehicle simulator, fast group	Speed, engine speed, gear, throttle, lap time, DRS, ERS, intervention state	10 Hz, changed values only
Vehicle simulator, slow group	Fuel, coolant, odometer, tyre pressures, heart rate	1 Hz
Vehicle simulator, static	Units, performance mode, lamp states	Once at start
Driver monitor	Fatigue, distraction, attention, eyes-on-road, active, alert state	10 Hz publish, 15 Hz processing

5.3 Signal contract

The signal contract is the set of VSS paths a consumer may rely on. It is listed in appendix A and is the only coupling between producers and consumers. A signal contract test (sprint 2) asserts that every path in appendix A resolves in the compiled tree with the expected datatype and unit.

5.4 Worked example: one signal, bottom to top

Follow one value, a road speed of 87.4 km/h, from the wire to the dial. It travels as `Vehicle.Speed`, which appendix A defines as a `float` in km/h. Today the simulator enters at step 3; steps 1 and 2 are the sprint 3 CAN provider.

Step	What happens	What the value is here
1	The powertrain ECU puts the frame carrying `PT_VehicleSpeed` on the bus. SocketCAN hands the raw bytes to user space	Raw bus bytes, meaningless on their own
2	The provider applies the scale and offset the DBC defines for that signal, then maps it to a catalogue path. The mapping and its 100 ms interval are in `vss/agl_vss_overlay.vspec`	A number, 87.4, now named `Vehicle.Speed`
3	The provider publishes to the broker over gRPC. Today `emulator/telemetry_sim.py` sets the same path at 10 Hz instead	`Vehicle.Speed` = 87.4
4	The broker checks it against the catalogue entry. A wrong type or an out-of-range value is refused and never reaches a consumer (IR-2, CS-5)	`float`, km/h, in range, accepted
5	Every open subscription is pushed the value with its timestamp	Flutter and Compose both redraw the dial

Only steps 1 and 2 know that CAN exists. From step 3 on the value is just a named number, and a consumer cannot tell whether it came from a bus, the simulator or a replayed log. Note also where the protocols stop: gRPC exists only above the line, and below it there is no RPC at all.

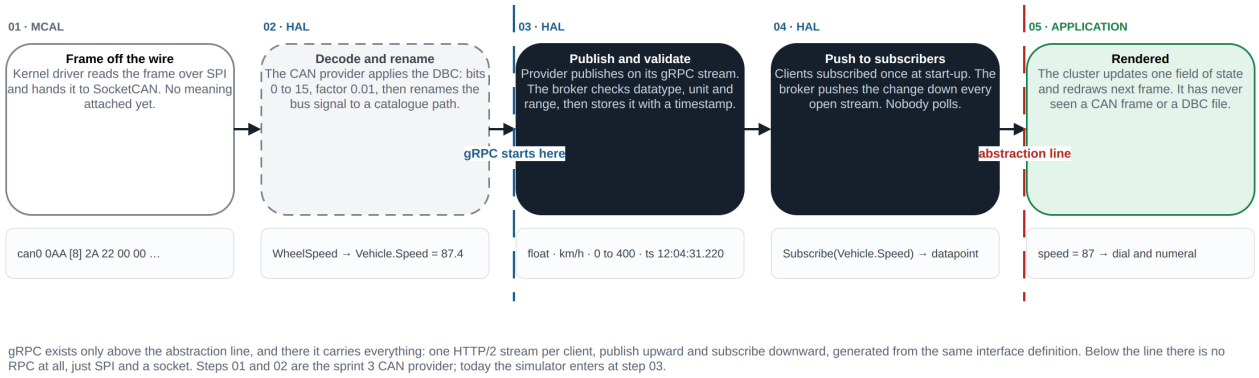

Figure 3: One signal from the wire to the dial. Steps 01 and 02 are the sprint 3 CAN provider; today the simulator enters at step 03.

Source: `docs/diagrams/signal-path.drawio`.

6. Runtime view

One turn of the loop, repeated at the broker tick:

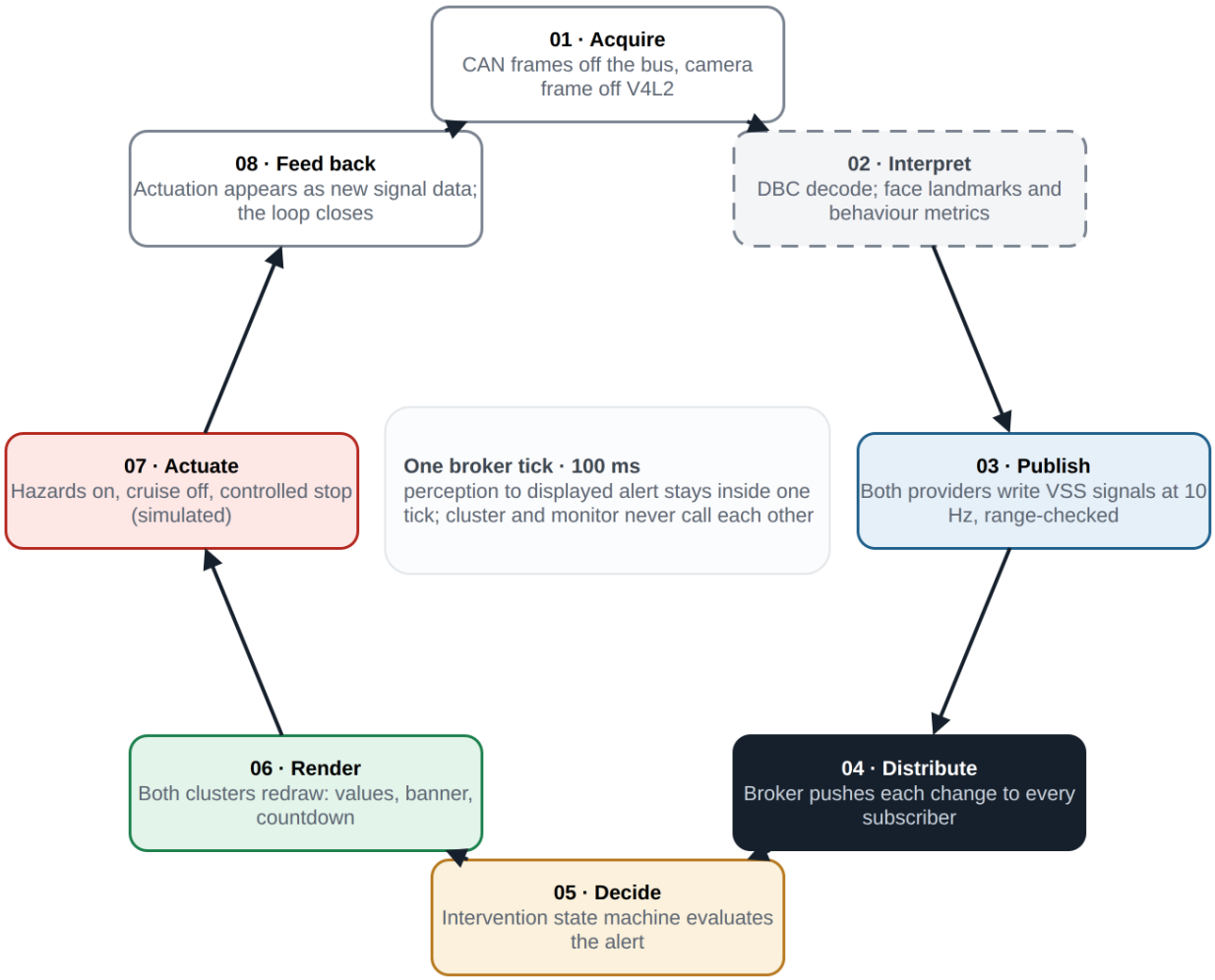

Figure 4: One turn of the cycle. It never stops turning, and no central coordinator drives it.

Source: `docs/diagrams/runtime-loop.drawio`.

Step	Name	What happens
01	Acquire	CAN frames off the bus (simulator today), camera frame off V4L2
02	Interpret	DBC decode on the vehicle side; face landmarks and behaviour metrics on the driver side
03	Publish	Both providers write VSS signals at 10 Hz; the broker range-checks each value against the catalogue
04	Distribute	The broker pushes each change down every open subscription stream; no provider knows who listens
05	Decide	The simulator's intervention state machine evaluates the alert state
06	Render	Both clusters redraw: values, warnings, the alert banner, the countdown
07	Actuate	Hazards on, cruise disengaged, controlled braking to standstill (simulated)
08	Feed back	The actuation appears as new signal data and the loop closes without a central coordinator

Perception to displayed alert stays inside one 100 ms broker tick. Cluster and monitor never call each other.

6.1 Minimum-risk manoeuvre state machine

Held in the simulator's cruise mode, driven only by `Vehicle.Driver.Monitoring.AlertState`:

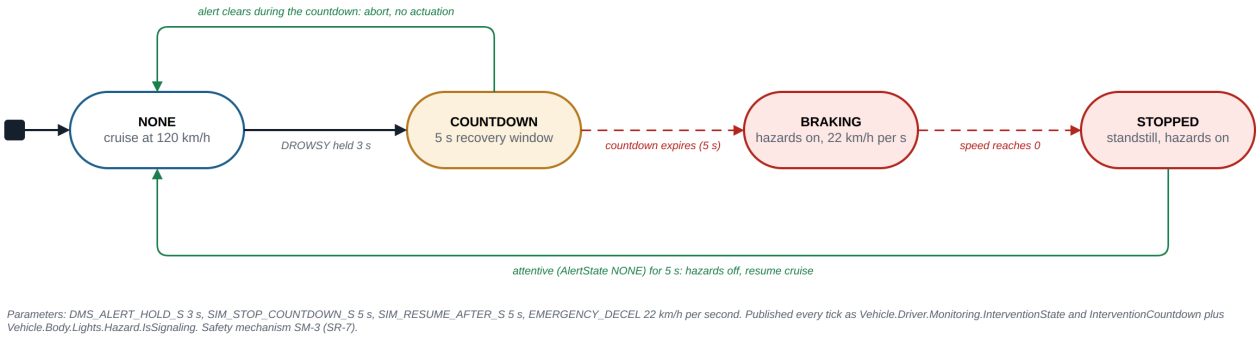

Figure 5: The minimum-risk manoeuvre state machine with its current parameters.

Source: `docs/diagrams/mrm-state-machine.drawio`.

State	Entry	Exit
NONE	Start, or countdown aborted, or resume complete	AlertState becomes DROWSY: go to COUNTDOWN
COUNTDOWN	DROWSY sustained past the 3 s alert hold	AlertState leaves DROWSY: back to NONE. Countdown of 5 s expires: go to BRAKING with hazards on
BRAKING	Countdown expired	Speed reaches 0: go to STOPPED
STOPPED	Standstill	AlertState NONE for 5 s: hazards off, resume cruise, back to NONE

The specification of this mechanism as a safety mechanism, with its parameters, is PULSE-SAF-001 SM-3.

7. Deployment view

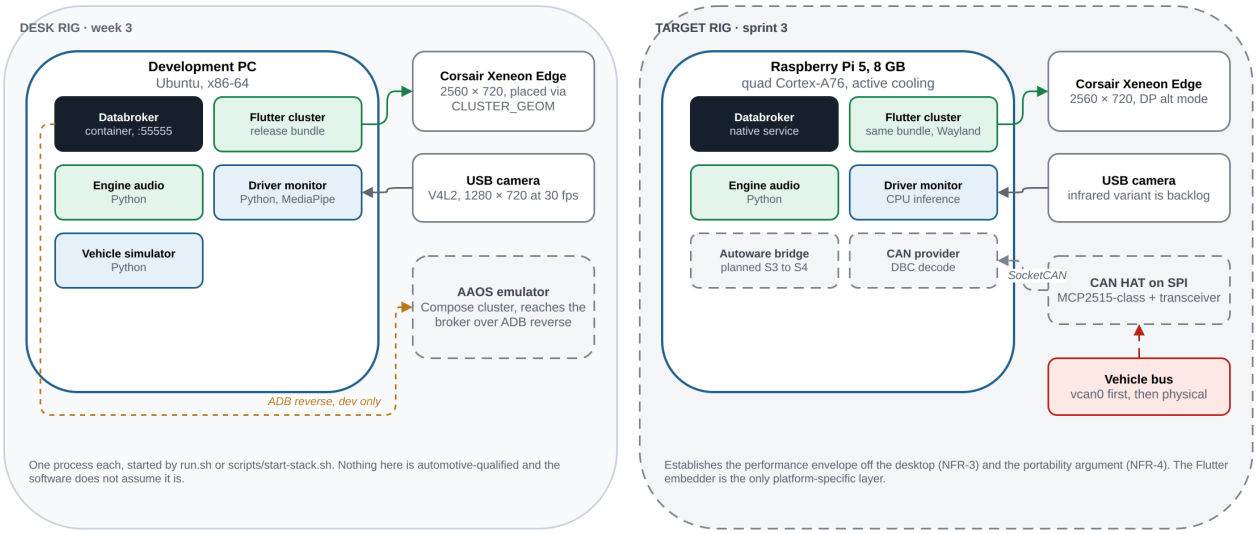

Figure 6: The desk rig at week 3 and the sprint 3 target rig. The same Flutter bundle runs on both; the embedder is the only platform-specific layer.

Source: `docs/diagrams/deployment.drawio`.

7.1 Desk rig, week 3

Node	Hosts	Notes
Development PC, Ubuntu, x86-64	Databroker (container), simulator, driver monitor, engine audio, Flutter cluster	One process each; started by `run.sh` or `scripts/start-stack.sh`
Corsair Xeneon Edge	Flutter cluster surface	2560 x 720, DisplayPort alt mode; located at runtime
USB camera	Driver monitor input	1280 x 720 at 30 fps
Android Automotive emulator	Compose cluster	Reaches the host broker through ADB reverse

7.2 Target rig, sprint 3

Node	Hosts	Notes
Raspberry Pi 5, 8 GB	The whole Linux stack	Establishes the performance envelope off the desktop (NFR-3)
CAN HAT on SPI	MCP2515-class controller with transceiver	Exposed as SocketCAN; `vcan0` first
Corsair Xeneon Edge, USB camera	As above	

Nothing on either rig is automotive-qualified, and the software does not assume it is.

8. Design decisions and deviations

Recorded here because PULSE-SAF-001 SR-10 requires every decision that could block a later ASIL argument to be written down with its rationale.

ID	Decision	Rationale	Consequence for a production programme
DD-1	HMI toolkit: Flutter on Linux as the primary path	AGL's cluster toolkit of choice; open source; one engineer can reach it	No safety-qualified Flutter renderer exists; a production cluster would need a qualified renderer or a supervisor that verifies the drawn output (SR-6)
DD-2	Second HMI: Jetpack Compose on Android Automotive	Answers the sponsor's question with a measured pair	AAOS platform footprint is 25x the Linux path; Linux stays primary
DD-3	Broker: Eclipse KUKSA databroker in a container	Reference implementation of the VSS broker; used by AGL	Container runtime is a desk convenience; target deployment runs it as a native service
DD-4	API version: `kuksa.val.v1`	Matches AGL and its overlay	Migration to v2 when AGL moves; stubs regenerate from the protos
DD-5	Signal catalogue pinned to VSS 6.0	Constraint C-2	Upgrades are reviewed changes; the AGL overlay must be re-validated per version
DD-6	Development transport: plaintext gRPC on loopback (`--insecure`)	Desk rig, one host	Removed by CS-3 in sprint 2; start-up warning until then
DD-7	Window system on the desk rig: X11 with `xrandr` placement	Development host convenience	Target uses Wayland; the runner reads `CLUSTER_GEOM` so placement logic does not change
DD-8	Driver monitor runs on CPU, no accelerator	6.5 ms per frame is already inside budget; avoids hardware dependence (A-2)	A production DMS would still need an infrared camera and a qualified evaluation set (backlog)
DD-9	Safety-loop actuation lives in the simulator	No vehicle exists; the manoeuvre must be demonstrable	On a real vehicle the actuation moves to a motion controller; the trigger signal contract stays
DD-10	Databroker image consumed at tag `latest`	Convenience at project start	Closed: pinned to the 0.7.0 tag and digest in `scripts/start-databroker.sh`
DD-11	Flutter SDK version not pinned	Convenience at project start	Violates NFR-8; pin via FVM or a recorded version in sprint 2

9. Verification approach

Level	What	When
Signal contract test	Every path in appendix A resolves in `vss_agl.json` with the expected type and unit; overlay paths present	Sprint 2, then every demo
Provider range test	Simulator and driver-monitor outputs stay inside catalogue ranges over a full cycle	Sprint 2
Broker smoke test	Broker up, tree served, a subscribe stream receives updates within one tick	Sprint 2, then every demo
UI smoke test	Each HMI launches, connects, and renders a known value	Sprint 2
HMI purity check	No CAN or DBC token in HMI source (FR-11)	Sprint 2
Benchmark	CPU, memory, artefact size under the replayed cycle, both HMIs	Sprint 4
Clean-machine bring-up	From README and ONBOARDING.md only	Done 2026-08-18; repeated at handover

Appendix A. Signal contract

A.1 Signals consumed by the HMIs

VSS path	Datatype	Unit	Source	Notes
Vehicle.Speed	float	km/h	Simulator	Primary value
Vehicle.Powertrain.CombustionEngine.Speed	uint16	rpm	Simulator	Redline band, shift ghost
Vehicle.Powertrain.Transmission.SelectedGear	int8		Simulator	-1 = R, 0 = N, 126 = P, 127 = D

VSS path	Datatype	Unit	Source	Notes
Vehicle.Powertrain.Transmission.PerformanceMode	string		Simulator	Allowed values uppercase
Vehicle.Powertrain.FuelSystem.RelativeLevel	uint8	percent	Simulator	
Vehicle.Powertrain.FuelSystem.Range	uint32	m	Simulator	
Vehicle.TraveledDistance	uint32	m	Simulator	Odometer
Vehicle.Chassis.Axle.Row1.Wheel.Left.Tire.Pressure	uint16	kPa	Simulator	Front left
Vehicle.Chassis.Axle.Row1.Wheel.Right.Tire.Pressure	uint16	kPa	Simulator	Front right
Vehicle.Chassis.Axle.Row2.Wheel.Left.Tire.Pressure	uint16	kPa	Simulator	Rear left
Vehicle.Chassis.Axle.Row2.Wheel.Right.Tire.Pressure	uint16	kPa	Simulator	Rear right
Vehicle.Body.Lights.Hazard.IsSignaling	boolean		Simulator	Set during the manoeuvre
Vehicle.Driver.HeartRate	uint16	bpm	Simulator	
Vehicle.Driver.AttentiveProbability	float	percent	Driver monitor	
Vehicle.Driver.FatigueLevel	float	percent	Driver monitor	
Vehicle.Driver.DistractionLevel	float	percent	Driver monitor	
Vehicle.Driver.IsEyesOnRoad	boolean		Driver monitor	
Vehicle.Driver.Monitoring.IsActive	boolean		Driver monitor	Overlay
Vehicle.Driver.Monitoring.AlertState	string		Driver monitor	Overlay; NONE, DROWSY, DISTRACTED, NO_FACE
Vehicle.Driver.Monitoring.InterventionState	string		Simulator	Overlay; NONE, COUNTDOWN, BRAKING, STOPPED
Vehicle.Driver.Monitoring.InterventionCountdown	uint8	s	Simulator	Overlay; 0 when idle
Vehicle.Motorsport.LapNumber	uint16		Simulator	Overlay
Vehicle.Motorsport.LapTime.Current	float	s	Simulator	Overlay
Vehicle.Motorsport.LapTime.Best	float	s	Simulator	Overlay; 0 until a lap completes
Vehicle.Motorsport.DRS.IsAvailable	boolean		Simulator	Overlay
Vehicle.Motorsport.DRS.IsActive	boolean		Simulator	Overlay
Vehicle.Motorsport.ERS.Level	uint8	percent	Simulator	Overlay; max 100
Vehicle.Motorsport.ERS.State	string		Simulator	Overlay; DEPLOY, HARVEST, IDLE

A.2 Overlay files

File	Adds	Origin
`vss/agl_vss_overlay.vspec`	AGL's cockpit signals: steering-wheel switches, infotainment, navigation, acceleration, angular velocity	AGL meta-agl-demo, `vss-agl_6.0`
`vss/pulse_vss_overlay.vspec`	`Vehicle.Motorsport` branch: lap number, lap times, DRS, ERS	PULSE
`vss/dms_vss_overlay.vspec`	`Vehicle.Driver.Monitoring` branch: active, alert state, intervention state, countdown	PULSE

A.3 Regeneration

The compiled tree is regenerated from the pinned VSS 6.0 specification and the three overlays by the command in the README section "Regenerate the VSS JSON". Regeneration has been confirmed byte-identical (FR-3).